Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl

SANTO TOMÁS EDUCACIÓN CONTINUA

1. Estructuras de Datos Primitivas y Colecciones

- **Tipos Básicos:** `int`, `float`, `str`, `bool`.
- **Listas:** Secuencias mutables ordenadas con índice base 0.
- **Diccionarios:** Pares clave-valor ideales para modelar registros estructurados.

```
# Definici n de estructuras clave en Python
meses = ['Ene', 'Feb', 'Mar']
kpi_dict = {'nombre': 'EBITDA', 'valor': 25000}
print(meses[0])              # Salida: Ene
print(kpi_dict['nombre'])    # Salida: EBITDA
```

2. Control de Flujo y Automatización Lógica

- Las sentencias condicionales (`if-elif-else`) permiten tomar decisiones basadas en criterios cuantitativos.
- Los bucles (`for`, `while`) automatizan procesos iterativos masivos.

```
# Evaluaci n condicional de m rgenes financieros
margen = 0.28
if margen >= 0.25:
    estado = "Objetivo Cumplido"
elif margen >= 0.15:
    estado = "Revisi n Moderada"
else:
    estado = "Alerta Cr tica"
```

3. Carga y Estructuración con Pandas (DataFrames)

- El **DataFrame** es el equivalente programático a una hoja de cálculo relacional.
- Permite leer archivos tabulares pesados de forma rápida y eficiente.

```
import pandas as pd

# Carga de transacciones y auditor a inicial
df = pd.read_csv('ventas.csv', sep=';')
print(df.head())
print(df.info())
```

4. Limpieza y Tratamiento de Valores Nulos

- Las fuentes de datos reales contienen valores faltantes (`NaN`) que corrompen cálculos numéricos.
- Es vital identificar y rellenar o eliminar anomalías de forma controlada.

```
# Detecci n e imputaci n de valores faltantes
print(df.isna().sum())

# Rellenar nulos con la mediana corporativa
mediana_presupuesto = df['presupuesto'].median()
df['presupuesto'] = df['presupuesto'].fillna(mediana_presupuesto)
```

5. Operaciones Vectorizadas y Filtros Lógicos

- El uso de bucles explícitos procesa filas lentamente; las operaciones vectorizadas operan sobre columnas completas al instante.
- Se emplean máscaras booleanas para segmentar carteras o transacciones.

```
# C lculo vectorizado de desviaci n y filtrado por condici n
df['Desviacion'] = (df['Real'] - df['Presupuesto']) / df['Presupuesto'] * 100
df_criticos = df[df['Desviacion'] > 15.0]
```

6. Modelado Relacional con pd.merge()

- Permite unificar tablas de hechos y dimensiones mediante llaves primarias y foráneas (esquema en estrella).
- Equivale a uniones relacionales (SQL) o cruces avanzados corporativos.

```
# Cruce relacional entre transacciones y sucursales
df_consolidado = pd.merge(
    left=df_transacciones,
    right=df_sucursales,
    on='ID_Sucursal',
    how='left'
)
```

7. Agrupaciones y Tablas Dinámicas

- El patrón *Split-Apply-Combine* permite resumir información por categorías.
- La función $\texttt{pivot}_t able replica tablas dinámicas tradicionales de Excel$.

```
# Resumen ejecutivo agrupado por gerencia
resumen = df.groupby('Gerencia').agg(
    Ventas_Totales=('Ventas', 'sum'),
    Margen_Promedio=('Margen', 'mean')
).reset_index()
```

8. Visualización Interactiva con Plotly

- Plotly Express permite transformar DataFrames en gráficos interactivos con tooltips y zoom nativo.
- Facilita la exportación autónoma a formatos web (`.html`).

```
import plotly.express as px

fig = px.bar(df, x='Mes', y='Ventas', color='Gerencia',
             title='Ventas por Gerencia y Mes')
fig.write_html('dashboard_ejecutivo.html')
```

9. Contenido para Profundizar en Analítica

- **Automatización del Sistema Operativo:** Uso de librerías como `os`, `subprocess` y la integración con programadores de tareas (Task Scheduler o Cron) para despliegues desatendidos.
- **Conectividad con Bases de Datos:** Consulta directa a motores relacionales mediante `SQLAlchemy` y `psycopg2` (PostgreSQL/SQL Server).
- **Desarrollo Web Interactivo:** Creación de aplicaciones analíticas autónomas orientadas a gerencias utilizando **Streamlit** o Dash.
- **Modelado Predictivo Básico:** Introducción a técnicas de regresión y clasificación comercial mediante **Scikit-Learn**.

10. Mejores Prácticas y Calidad de Código

- **Separación de Credenciales:** Nunca almacene contraseñas o tokens en texto plano dentro del código fuente; delegue claves a variables de entorno del sistema operativo (`os.getenv()`).
- **Optimización de Memoria:** Convierta columnas categóricas de alta repetición al tipo `category` en Pandas para reducir drásticamente el consumo de RAM corporativa.
- **Control de Errores Robustos:** Implemente bloques de control estructurado (`try-except`) en procesos ETL y de envío automatizado de correos para prevenir interrupciones críticas.
- **Modularidad y Legibilidad:** Organice los flujos analíticos en funciones reutilizables y documente las reglas de negocio críticas directamente en el código.